qaptiva-mbl

Analog spin-chain dynamics and ED benchmarking

Minimal user guide | v0.1.0 | October 2026

1 Purpose and scope

This package studies real-time dynamics in the **Imbrie** and **random-field XXZ** chains. A user selects physical parameters, boundary conditions and an initial product state; the package preserves the exact disorder realization, runs a Qaptiva `AnalogQPU` job, retrieves site-resolved observables and optionally compares the result against independent **exact diagonalization (ED)** using NumPy/SciPy.

User setting	Available choices
Model	`imbrie` or `xxz`
Geometry	Chain length L; `obc` or `pbc`
Initial state	Néel state or random Z-basis product state
Disorder	User-supplied seed and model couplings
QPU integration	Adaptive error control or a fixed number of steps
Outputs	Local autocorrelators, site average, ED comparison and error plots

The initial release supports *one disorder realization per run.* There is no automatic disorder averaging, entanglement calculation or $U(1)$ symmetry-sector reduction.

2 Hamiltonians and normalization

Sites are labeled $j = 0, 1, \ldots, L-1$, and we set $\hbar = 1$. Let $X_j = \sigma_j^x$, $Y_j = \sigma_j^y$, and $Z_j = \sigma_j^z$ be Pauli operators. Neighbor pairs are

$$\mathcal{B}_{\mathrm{OBC}} = \{(j, j+1) \mid 0 \le j < L-1\}, \qquad \mathcal{B}_{\mathrm{PBC}} = \mathcal{B}_{\mathrm{OBC}} \cup \{(L-1, 0)\}. \tag{1}$$

OBC requires $L \ge 2$; PBC requires $L \ge 3$ in this implementation.

2.1 Imbrie model: Pauli convention

$$\boxed{H_{\mathrm{I}} = \sum_{(i,j)\in\mathcal{B}} \Delta_{ij} Z_i Z_j + \sum_{j=0}^{L-1} \big(h_j Z_j + \Gamma X_j\big).} \tag{2}$$

The bond couplings and longitudinal fields are independent draws:

$$\Delta_{ij} \sim \mathcal{U}(\Delta_{\min}, \Delta_{\max}), \qquad h_j \sim \mathcal{U}(-h_0, h_0). \tag{3}$$

The Hamiltonian uses Pauli matrices *directly*, without extra factors of $1/2$. Configuration keys: `gamma`, `delta_min`, `delta_max`, `h0`.

2.2 Random-field XXZ: spin-$\frac{1}{2}$ convention

With $S_j^\alpha = \sigma_j^\alpha/2$ and $h_j \sim \mathcal{U}(-h_0, h_0)$,

$$\begin{aligned} H_{\mathrm{XXZ}} &= \sum_{(i,j)\in\mathcal{B}} \left[J_{xy}(S_i^x S_j^x + S_i^y S_j^y) + J_z S_i^z S_j^z\right] + \sum_j h_j S_j^z \\ &= \sum_{(i,j)\in\mathcal{B}} \left[\frac{J_{xy}}{4}(X_iX_j + Y_iY_j) + \frac{J_z}{4} Z_iZ_j\right] + \frac{1}{2}\sum_j h_j Z_j. \end{aligned} \tag{4}$$

The *second line* specifies the exact Pauli coefficients used for both solvers. Configuration keys: `jxy`, `jz`, `h0`.

3 Initial states and measured observables

The package prepares a product eigenstate of all Z_j,

$$|\psi_0\rangle = |s_0, s_1, \ldots, s_{L-1}\rangle, \qquad Z_j|\psi_0\rangle = s_j|\psi_0\rangle, \qquad s_j \in \{+1, -1\}. \tag{5}$$

- **Néel:** `initial_state="neel"`, $s_j = (-1)^j$.
- **Random:** `initial_state="random"`, independent $s_j = \pm 1$.

In the Qaptiva input bitstring, $s_j = +1$ maps to `0` and $s_j = -1$ to `1`. **Site 0 is the leftmost bit.** Independent deterministic pseudorandom streams separate Hamiltonian disorder from random-state selection.

For a time-independent Hamiltonian, $|\psi(t)\rangle = e^{-iHt}|\psi_0\rangle$. Define $Z_j(t) = e^{iHt} Z_j e^{-iHt}$. The product-state site-resolved autocorrelator is

$$\boxed{C_j(t) = \langle\psi_0|Z_j(t)Z_j|\psi_0\rangle = s_j\langle\psi(t)|Z_j|\psi(t)\rangle,} \qquad \boxed{C(t) = \frac{1}{L}\sum_{j=0}^{L-1} C_j(t).} \tag{6}$$

The QPU job measures unweighted local magnetizations $\langle Z_j(t)\rangle$. After retrieval, the package multiplies each site by the *stored* initial spin s_j. For ideal unitary dynamics $C_j(0) = 1$.

Interpretation: for one product state, $C_j(t)$ is *not* the infinite-temperature trace correlation $2^{-L}\,\mathrm{Tr}[Z_j(t)Z_j]$; the latter requires a trace or suitable state averaging.

4 Solver settings: adaptive or fixed-step

The `RunConfig` constructor enforces **mutually exclusive** integration settings:

Adaptive	`error_control=True`, `nsteps=None`; provide positive `abs_tol` and `rel_tol`. Qaptiva selects internal steps through its adaptive Dormand–Prince Runge–Kutta (DP5) method.
Fixed steps	`error_control=False`; supply a positive integer `nsteps`. Adaptive tolerances are not forwarded to the remote solver.

```
# Adaptive (default): do not specify nsteps
error_control=True, nsteps=None, abs_tol=1e-8, rel_tol=1e-8

# Fixed-step alternative
error_control=False, nsteps=1000
```

The *internal integration steps* are not the same as the *returned observation times*. The package always takes timestamps from the QPU result. The `use_gpu` setting controls the backend's GPU option where available.

5 Notebook as the main interface

Run `notebooks/MBL_Simulation.ipynb`. Its cells perform configuration, sample preparation, optional submission, optional retrieval, ED evolution, plotting and comparison. The remote cells start *disabled*; enabling them requires a configured Qaptiva installation.

6 Installation

Requires **Python 3.10+**, NumPy, SciPy and Matplotlib. Run these commands from the repository root:

```
python -m pip install -e ".[notebook,test]"
python -m pytest -q
jupyter notebook notebooks/MBL_Simulation.ipynb
```

The Qaptiva/QLMaaS modules `qat` and `qlmaas`, together with authenticated server access, are required only for remote submission/retrieval; they are not bundled in this repository. ED-only workflows run locally.

7 Prepare, submit and retrieve

Use the notebook, or call the public API:

```
from qaptiva_mbl import RunConfig, prepare_run
from qaptiva_mbl import submit_run, retrieve_run

cfg = RunConfig(
    model="imbrie", L=8, boundary="pbc", seed=12345,
    initial_state="neel", gamma=1.0,
    delta_min=0.8, delta_max=1.2, h0=3.5,
    tmax=100.0, error_control=True, nsteps=None,
    abs_tol=1e-8, rel_tol=1e-8, use_gpu=False,
)
run_dir = prepare_run(cfg, root="runs")
job_id = submit_run(run_dir)   # requires Qaptiva access
print(job_id)

# Retrieve now or in a later session from the same run_dir.
times, C_qpu = retrieve_run(run_dir, host="YOUR-QLMAAS-HOST")
```

For the XXZ chain, change `model="xxz"` and specify `jxy` and `jz`. The package persists the random fields, Imbrie bonds (if present), initial spins, parameters and remote job ID. It never needs to redraw a disorder sample at retrieval time.

8 Independent manual ED

ED constructs the same spin Hamiltonian as a sparse matrix in the *full* 2^L-dimensional basis, without using Qaptiva's matrix constructors. The time evolution uses

$$|\psi(t)\rangle = e^{-iHt}|\psi_0\rangle. \tag{7}$$

The `dense` method diagonalizes H once, while `krylov` applies sparse matrix exponentials. With `method="auto"`, dense ED is used for $L \leq 9$ and Krylov for larger L. The default `max_L=12` safety limit can be raised explicitly, at increased computational cost.

```
from qaptiva_mbl import run_ed, compare_run

# Reuse the exact QPU timestamps, saved disorder and initial state.
times, C_ed = run_ed(run_dir, max_L=12)
metrics = compare_run(run_dir)
print(metrics["global_max_absolute_error"])
```

For **ED without Qaptiva**, prepare a run and supply a time array:

```
import numpy as np
from qaptiva_mbl import RunConfig, prepare_run, run_ed
run_dir = prepare_run(RunConfig(model="xxz", L=8))
times, C_ed = run_ed(run_dir, times=np.linspace(0, 100, 201))
```

9 Quantifying the ED–AnalogQPU difference

For the *same* Hamiltonian realization, initial state and returned timestamps t_k, define

$$\delta C_j(t_k) = C_j^{\mathrm{QPU}}(t_k) - C_j^{\mathrm{ED}}(t_k), \qquad \epsilon_j(t_k) = |\delta C_j(t_k)|. \tag{8}$$

The analysis also computes the site-resolved average difference and three nonnegative aggregate errors:

$$\delta C(t_k) = \frac{1}{L}\sum_{j=0}^{L-1} \delta C_j(t_k), \tag{9}$$

$$\mathrm{MAE}(t_k) = \frac{1}{L}\sum_{j=0}^{L-1} |\delta C_j(t_k)|, \tag{10}$$

$$\mathrm{RMSE}(t_k) = \sqrt{\frac{1}{L}\sum_{j=0}^{L-1} [\delta C_j(t_k)]^2}, \tag{11}$$

$$\epsilon_{\max}(t_k) = \max_{0\le j<L} |\delta C_j(t_k)|. \tag{12}$$

A global maximum, $\max_{j,k} |\delta C_j(t_k)|$, global RMSE and per-site maximum errors are also saved. Note that positive and negative deviations can cancel in $\delta C(t)$; it is not interchangeable with MAE. These metrics quantify *solver agreement*, not extrapolation to the thermodynamic limit.

10 Plots and results directory

After retrieving QPU data, figures can be regenerated without any network call:

```
from qaptiva_mbl import plot_run
plot_run(run_dir, kind="sites", sites=[0,1,2,3],
         save="sites.pdf")
plot_run(run_dir, kind="average", save="average.pdf")
plot_run(run_dir, kind="error", save="errors.pdf")
plot_run(run_dir, kind="site_error", sites=[0,1],
         save="site_errors.pdf")
```

Optional arguments include `log_time=True` and `overlay_ed=True`. The function returns a Matplotlib figure for customization. In ED-only mode, the notebook plots the saved ED arrays directly.

```
runs/imbrie_L8_<unique-id>/
    run.json          # run configuration, bitstring, job ID, status
    sample.npz        # exact fields, bond couplings, initial spins
    qpu_results.npz   # time, local magnetization and correlators
    ed_results.npz    # time and ED correlators
    comparison.npz    # signed/absolute differences and summaries
```

Only files produced by completed stages exist. The result arrays are indexed as $[j, k]$ for site j and observation time t_k.

11 Reliability notes and status

- **Time alignment:** comparison rejects mismatched observation grids. ED must use the stored QPU times.
- **Observable order:** site 0 is read from `value_data`; remaining sites are read from `values_data` in order.
- **Reproducibility:** full-precision random arrays and the initial bitstring are saved before submission; job metadata is retained in `run.json`.
- **Security:** never store credentials in source, notebooks, run metadata or Git.
- **Validation status:** local tests and mock retrieval pass; check live Qaptiva submission and retrieval against the intended server.

References: Qaptiva analog simulation overview | Getting started | Schedules.
Project: https://github.com/abdpdn/qaptiva-mbl